"Deep Learning Vier Gewinnt" - technische Dokumentation

Die folgende technische Dokumentation beschreibt den Aufbau und die Implementierung des im Rahmen meiner Facharbeit entwickelten Systems. Sie richtet sich an Leser mit bestehenden Programmierkenntnissen und dient als Referenz für die im Hauptteil beschriebenen Konzepte und Ergebnisse.

Das System wurde vollständig in Python implementiert und nutzt PyTorch als Framework für das neuronale Netz, NumPy für die Zustandsrepräsentation des Spielfelds und weitere mathematische Berechnungen sowie Pandas und Matplotlib für die Auswertung der Trainingsdaten.

1. Übersicht

1.1 UML Klassendiagramm 2

2. Dokumentation

2.1 VierGewinnt 3

2.2a LinearDQN 4

2.2b ConvDQN 5

2.3 Agent 6-7

2.4 Episode 8

2.5 Session 9

2.6 Trainer 10-11

2.7 Logger 12

2.8 Training 13

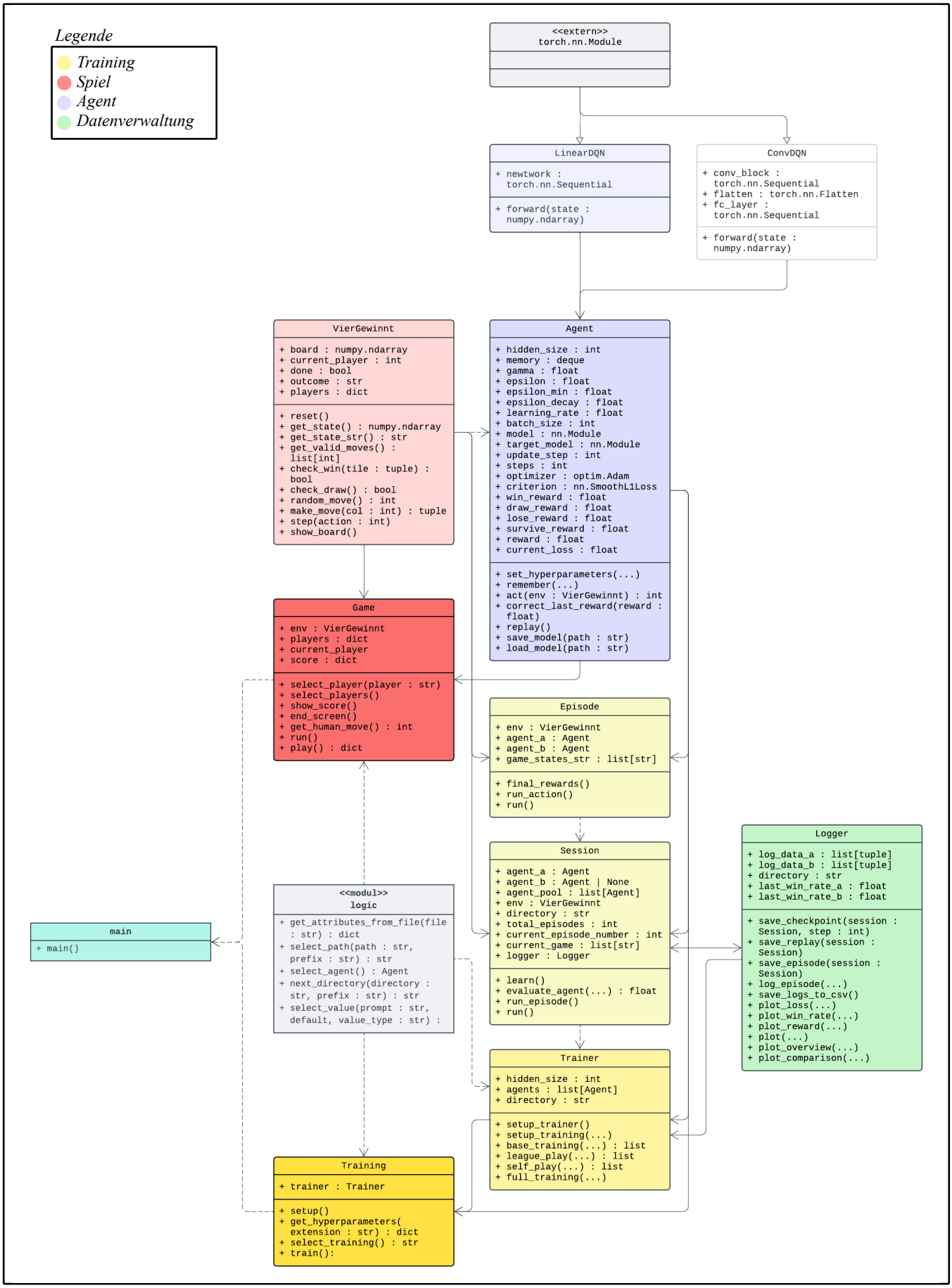
2.9 Game 14

2.10 logic 15

2.11 main 16

3. Quellen

1.1 UML Klassendiagramm



Klammern welche "..." enthalten stellen ausgelassene Parameter dar (meistens ausgelassen, da zu lang). ConvDQN nicht implementiert. Wenn hinter einer Methode ":" ohne Parameter steht, gibt es mehrere Rückgabewerte. Gestrichelte Linien stellen eine Abhängigkeit und durchgezogene eine Assoziation dar. Geschlossene Pfeilspitzen stellen eine Vererbung dar.

2.1 VierGewinnt

Die VierGewinnt Klasse ist das eigentliche Spiel bzw. die Trainingsumgebung. Sie verwaltet die Spielregeln, Spielzustände, welche Züge gemacht werden etc..

Klassen-Struktur & Initialisierung

Die Klasse nutzt die numpy Bibliothek, um das Spielfeld effizient als Matrix zu verarbeiten. Die Spielfeldzustände können entweder 0 (freies Feld), 1 (Spieler 1) oder -1 (Spieler 2) sein.

`__init__(self):`

Erzeugt ein leeres Spielfeld der Größe 6x7 (Zeilen x Spalten)

Bestimmt per Zufall (numpy.random.choice), welcher Spieler (1 oder -1) beginnt

Initialisiert Variablen wie done (Spiel beendet?) und outcome (Ergebnis

Zentrale Methoden

`get_state(self)  -> numpy.ndarray`

Gibt eine Kopie des aktuellen Spielfelds zurück. Die Rückgabe einer Kopie (.copy()) verhindert, dass das Spielbrett irgendwo außerhalb der VierGewinnt Klasse verändert wird.

`get_valid_moves(self) -> list[int]`

Analysiert die oberste Zeile (Index 0) des Boards. Jede Spalte, deren oberstes Feld den Wert 0 hat, ist noch nicht voll und wird als legaler Zug (Index 0-6) zurückgegeben.

`make_move(self, col : int) -> tuple[int, int] | None`

Diese Methode implementiert die "Physik". Sie iteriert von der untersten Zeile (Index 5) nach oben. Der Stein wird in das erste freie Feld der gewählten Spalte gesetzt. Die Koordinaten (row, col) des gesetzten Steins oder None, falls die Spalte voll ist werden ausgegeben, damit im folgenden Siegesbedingungen geprüft werden können.

`check_win(self, tile : tuple[int, int]) -> bool`

Die Siegprüfung erfolgt lokal ausgehend vom letzten gesetzten Stein (tile).
Geprüft wird in vier Richtungen: Horizontal, Vertikal und die beiden Diagonalen.
In jede Richtung wird "ausgestrahlt", um aufeinanderfolgende Steine desselben Spielers zu zählen.
Erreicht die Summe 4, wird True zurückgegeben.

`step(self, action : int)`

Dies ist die Hauptmethode für den Spielablauf:

Führt make_move mit der gewählten Spalte (action) aus.

Prüft mittels check_win, ob der Zug zum Sieg geführt hat.

Prüft mittels check_draw, ob das Feld voll ist.

Aktualisiert self.done und setzt das Ergebnis in self.outcome.

2.2a LinearDQN

Die LinearDQN Klasse definiert die Architektur des künstlichen neuronalen Netzes. Sie basiert auf dem torch.nn-Modul von PyTorch und stellt ein "Deep Q-Network" (DQN) dar, das darauf trainiert wird, die voraussichtliche Belohnung (Q-Wert) für jeden möglichen Spielzug vorherzusagen.

Klassen-Struktur & Initialisierung

Das Modell ist als klassisches "Feedforward"-Netzwerk (Multilayer Perceptron) aufgebaut. Es verarbeitet das zweidimensionale Spielfeld als eindimensionalen Eingabe-Vektor und transformiert diesen durch mehrere Schichten in eine Bewertung der sieben möglichen Spalten (Q-Value).

`__init__(self, hidden_size : int)`

Input Layer: Die Spielfeldgröße ist fest auf 6x7 Felder (42 Eingangsneuronen) eingestellt. Mittels nn.Flatten() wird die Matrix in einen eindimensionalen Vektor umgewandelt.

Hidden Layers: Es werden drei vollvernetzte Schichten (nn.Linear) verwendet. Die Anzahl der Neuronen pro Schicht wird über den Parameter hidden_size gesteuert, was Experimente mit unterschiedlicher Komplexität ermöglicht.

Aktivierungsfunktion: Zwischen den linearen Schichten wird nn.ReLU (Rectified Linear Unit) eingesetzt. Dadurch kann das Netzwerk komplexe taktische Muster und Zusammenhänge im Spiel lernen (Nähere Analyse im Huaptteil).

Output Layer: Die letzte Schicht hat 7 Ausgangsneuronen. Jedes Neuron korrespondiert mit einer Spalte des Spielfelds (0–6). Der ausgegebene Wert stellt den "Q-Wert" dar – also die Erwartung, wie gut ein Zug in diese Spalte ist.

Zentrale Methoden

`forward(self, state : numpy.ndarray)`

Dies ist die Kernmethode für die Vorwärtspropagierung der Daten:

Sie wird nicht explizit aufgerufen, sondern implizit genutzt, wenn das Modell mit Daten gespeist wird (self.model(state)).

Der aktuelle Spielzustand (state) wird durch alle Schichten des Netzwerks gereicht.

Rückgabe: Ein Tensor mit sieben Werten (Bewerungen).

2.2b ConvDQN

Das ConvDQN (Convolutional Deep Q-Network) ist eine fortschrittlichere Architektur, die speziell für die Verarbeitung von räumlichen Daten wie Spielfeldern oder Bildern entwickelt wurde. Im Gegensatz zum LinearDQN betrachtet es das Spielfeld nicht als flache Liste von Zahlen, sondern nutzt Faltungsschichten (Convolutional Layers), um geometrische Muster zu erkennen.

Für die finale Facharbeit ist ConvDQN nicht vollständig implementiert. Durch das von Grund aus gegebene geometrische "Verständnis", würde eine Implementierung helfen die im Training mit einem LinearDQN entstehenden Grenzen (siehe Hauptteil) zu überwinden. Dies würde jedoch den Rahmen der Facharbeit sprengen.

2.3 Agent

Die Klasse Agent bildet das Bindeglied zwischen dem neuronalen Netz (DQN) und der Spielumgebung. Sie implementiert die gesamte Logik des Reinforcement Learnings, verwaltet den Speicher des Agenten und steuert den Lernprozess sowie die Entscheidungsfindung (Aktionswahl).

Klassen-Struktur & Initialisierung

Der Agent speichert sowohl das aktuelle Arbeitsmodell als auch ein identisches "Target-Modell", um die Stabilität des Lernprozesses zu erhöhen. Zudem verwaltet er einen Erfahrungsspeicher (Replay Memory).

`__init__(self, ...)`

Modelle: Initialisiert zwei Instanzen des LinearDQN. Das target_model wird regelmäßig mit den Gewichten des Hauptmodells synchronisiert.

Replay Memory: Ein deque-Speicher mit einer Kapazität von 100.000 Erfahrungen. Hier werden vergangene Spielzüge gespeichert, um sie später in zufälligen Paketen (Batches) erneut zu lernen.

Hyperparameter:

gamma: Bestimmt die Gewichtung zukünftiger Belohnungen (0.95).
epsilon & epsilon_decay: Steuern die Explorationsrate (Zufall vs. Wissen).
learning_rate: Die Schrittweite des Optimierers (Adam) bei der Korrektur der Gewichte.
batch_size: Bestimmt wie groß das Paket an entnommenen Erinnerungen ist.

Belohnungsstruktur:

Definiert die Belohung für Sieg, Unentschieden, Niederlage und das bloße Überleben pro Zug).

Zentrale Methoden

`act(self, env : VierGewinnt) -> int`

Implementiert die "Epsilon-greedy" Strategie:

Mit einer Wahrscheinlichkeit von Epsilon wird ein zufälliger Zug gewählt (Exploration).

Ansonsten wird der Zustand an das Modell übergeben, welches die Q-Werte berechnet. Einer der Züge mit dem höchsten Wert wird gewählt (Exploitation).

Illegale Züge (volle Spalten) werden aktiv ausgefiltert, indem jene Q-Werte temporär auf die negative Unendlichkeit (-numpy.inf) geserztz wird.

`remember(self, state, action, reward, next_state, done)`

Speichert ein "Erfahrungstupel" im memory. Diese Daten bilden die Grundlage für das spätere Training.

`replay(self)`

Dies ist der eigentliche Lernalgorithmus (Training-Loop):

Batch-Sampling: Ein zufälliger Datensatz wird aus dem Speicher entnommen.

Loss-Berechnung: Der Agent berechnet die Differenz zwischen der aktuellen Vorhersage und dem Zielwert (basierend auf der Bellman-Gleichung: Siehe Hauptteil).

Optimierung: Mittels SmoothL1Loss und dem Adam-Optimierer werden die Gewichte des Netzwerks angepasst.

"Gradient Clipping": Begrenzt die Stärke der Gewichtsänderungen, um Instabilitäten zu vermeiden.

`correct_last_reward(self, new_reward : float)`

Zweck näher beschrieben in Episode

2.4 Episode

Die Klasse Episode beschreibt den Ablauf eines einzelnen, kompletten Spiels innerhalb des Lernprozesses. Sie steuert die Interaktion zwischen den Agenten und der Umgebung (VierGewinnt) für die Dauer einer Partie.

Klassen-Struktur & Initialisierung

Eine Episode verknüpft die Umgebung mit mindestens einem Agenten (agent_a). Optional kann ein zweiter Agent (agent_b) hinzugefügt werden, um "Self-Play" (Training gegen sich selbst) oder ähnliche Trainingsverfahren zu ermöglichen.

`__init__(self, env, agent_a, agent_b = None)`

Zuweisung: Speichert Referenzen auf die Umgebung und die beteiligten Agenten.

Protokollierung: Initialisiert die Liste game_states_str, in der der visuelle Verlauf des Spiels als Text gespeichert wird, was eine spätere Analyse einzelner Partien erlaubt.

Zentrale Methoden

`run_action(self)`

Die Methode steuert einen einzlenen Spielzug.

Sichtweisen-Normalisierung: Bevor ein Agent agiert, wird sichergestellt, dass er das Feld immer aus "seiner Sicht" sieht. Da Spieler 1 und Spieler -1 existieren, wird das Board für den zweiten Spieler mit -1 multipliziert. So lernt der Agent eine universelle Strategie, egal welcher Spieler er ist.

Entscheidungsfindung: Je nachdem, welcher Spieler am Zug ist, wird die act-Methode von agent_a, agent_b oder (falls kein zweiter Agent vorhanden ist) ein Zufallszug der Umgebung aufgerufen.

Erfahrung speichern: Nach dem Zug wird das Resultat (state, action, reward, next_state) mittels der remember-Methode im Gedächtnis des jeweiligen Agenten abgelegt.

`final_rewards(self)`

Diese Methode wird aufgerufen, sobald self.env.done wahr ist und verteilt die großen Belohnungen (Sieg, Niederlage, Unentschieden) rückwirkend an die Agenten.

"correct_last_reward" wird aufgerufen, um den letzten Zug im Gedächtnis des Agenten von einer "Überlebens-Belohnung" auf die tatsächliche "Final-Belohnung" zu korrigieren, da im Verlauf der Episode nach jedem Zug eine "Überlebens-Belohung" verteilt wird.

`run(self)`

Steuert den gesamten Spiel-Loop:

Setzt die Belohnungen der Agenten für die neue Runde auf 0.

Ruft in einer Schleife run_action auf, bis die Umgebung meldet, dass das Spiel beendet ist.

Führt nach Spielende final_rewards aus, um die finale Bewertung der Züge abzuschließen.

2.5 Session

Die Klasse Session bildet die übergeordnete Steuerungseinheit für einen Trainingslauf. Während eine Episode ein einzelnes Spiel kontrolliert, verwaltet die Session eine Sequenz von tausenden Spielen. Sie ist verantwortlich für das Steuern des Lernens, die Evaluation des Fortschritts und wann welche Daten protokolliert werden.

Klassen-Struktur & Initialisierung

Eine Session definiert den Rahmen des Trainings: Wer trainiert gegen wen und wie lange?

```
__init__(self, directory, total_episodes, agent_a, ...)
```

Agenten-Management: Verwaltet agent_a (den primären lernenden Agenten), optional einen agent_b (für Self-Play) oder einen agent_pool (eine Liste bereits trainierter Agenten als Gegner.

Umgebung: Erstellt eine eigene Instanz von VierGewinnt.

Tracking: Initialisiert den Logger, um Fortschritte im spezifizierten Verzeichnis (directory) zu speichern. Hält fest, bei welcher Episode (current_episode_number) sich das Training aktuell befindet.

Zentrale Methoden

```
evaluate_agent(self, agent, num_test_games=100) -> float
```

Dies ist die Benchmark-Methode des projekts:

Der Agent spielt 100 Partien gegen einen reinen Zufallsgegner. Während dieser Phase ist die Exploration deaktiviert (Epsilon = 0) , damit die wahre Spielstärke gemessen werden kann. Die Siegquote (Win-Rate) als Dezimalwert zwischen 0.0 und 1.0 wird zurückgegeben.

```
run_episode(self)
```

Steuert den Ablauf eines einzelnen Trainingsschritts innerhalb der Session:

Gegnerwahl: Entscheidet, ob der Agent gegen sich selbst, einen Agenten aus dem Pool oder (zu 10%) gegen einen Zufallsgegner spielt, um Overfitting zu vermeiden.

Lernaufruf: Ruft nach Ende der Episode die learn-Methode (Replay) auf, damit die Agenten ihre Erfahrungen verarbeiten.

Intervall-Check: Alle 100 Episoden wird evaluate_agent aufgerufen, um den aktuellen Leistungsstand statistisch zu erfassen.

```
run(self)
```

Die Hauptschleife der Session:

Iteriert von der ersten bis zur letzten geplanten Episode.

Ruft kontinuierlich run_episode auf.

Sorgt dafür, dass der Logger regelmäßig Checkpoints der Modellgewichte und der Spielabläufe speichert.

2.6 Trainer

Die Klasse Trainer bildet die höchste Organisationsebene des Projekts (vond er Benutzeroberfläche abgesehen). Sie fungiert als Verwalter für verschiedene Trainings-Szenarien und die langfristige Entwicklung der Agenten. Ihre Hauptaufgabe ist es, strukturierte Trainingsabläufe (wie das "League Training") zu ermöglichen, bei denen Agenten nicht nur gegen Zufall, sondern auch gegen immer stärkere Versionen ihrer selbst antreten.

Klassen-STruktur & Initialisierung

Der Trainer verwaltet eine Liste von Agenten (self.agents) und stellt sicher, dass alle Daten (Modellgewichte und Statistiken) konsistent in einer organisierten Ordnerstruktur gespeichert werden.

`__init__(self, hidden_size : int, ...)`

Agenten-Management: Erstellt den initialen Agenten mit der spezifizierten Netzwerkgröße (hidden_size).

Dateisystem: Generiert automatisch ein neues Verzeichnis (z. B. trainer_1), falls kein Name angegeben wird, um Überschreibungen von alten Trainingsläufen zu verhindern.

Zentrale Methoden

`setup_trainer(self)`

Erstellt, falls nötig, das Hauptverzeichnis für den Trainer.

Erzeugt eine Konfigurationsdatei (DQN_attributes), in der wichtige Parameter wie die hidden_size gespeichert werden. Dies ist wichtig, damit das Modell später mit der korrekten Architektur wieder geladen werden kann.

`base_training(self, episodes, ...)`

Startet ein Basistraining, bei dem der Agent gegen einen Zufallsgegner antritt.

Dies dient als "Grundschule", in der der Agent die grundlegenden Spielregeln und einfache Taktiken lernt und da keine anderen bereits trainierten Modelle erforderlich sind.

Rückgabe: Die gesammelten Statistiken (Loss, Win-Rate, etc.) als Liste für die spätere Visualisierung.

`league_play(self, episodes, agent_a, ...)`

Implementiert eine fortgeschrittene Trainingsform: Ein Agent tritt gegen einen "Pool" aus seinen Vorgänger-Versionen an.

Verhindert den "Overfitting", durch das ein Agent nur lernt, sehr effektiv, einen einzigen Gegnertyp zu besiegen.

Soll für eine kontinuierliche Steigerung der Spielstärke sorgen, da die Gegner im Pool ebenfalls (frühere) Top-Modelle sind.

`full_training(self, episodes, cycles, ...)`

Kombiniert base_training und mehrere Zyklen von league_play zu einem automatisierten Gesamtprozess.

Visualisierung: Am Ende wird automatisch eine Übersichtsgrafik (plot_overview) erstellt, die den gesamten Lernfortschritt über alle Phasen hinweg zeigt.

`self_play(self, episodes, agent_a, ...)`

Konzept: Es wird eine Session gestartet, in der sowohl agent_a als auch agent_b lernen.

Lerneffekt: Da beide Agenten gleichzeitig lernen, entsteht ein dynamisches Wettrüsten. Findet ein Agent eine neue Gewinnstrategie, muss der andere eine Verteidigung dagegen entwickeln.

Synchronisation: Nach dem Training werden die Modelle dem Agentenpool angehangen.

!!! self_play kann sehr instabil werden !!!

2.7 Logger

Die Klasse Logger ist für das Datenmanagement des Trainings zuständig. Sie protokolliert Leistungsdaten wie Loss, Belohnungen und Siegquoten und wandelt diese in aussagekräftige Grafiken um. Dies ist wichtig für die Auswertung.

Klassen-Struktur & Initialisierung

Der Logger arbeitet eng mit der Session zusammen und speichert Daten sowohl in Textform (CSV, TXT) als auch visuell (PNG, SVG).

`__init__(self, directory : str)`

Datenstrukturen: Initialisiert Listen (log_data_a, log_data_b), um die Metriken der Agenten während der Laufzeit zwischenzuspeichern.

Pfadverwaltung: Speichert den Pfad zum aktuellen Trainer-Verzeichnis, um Checkpoints und Grafiken korrekt zuzuordnen.

Zentrale Methoden

`save_checkpoint(self, session, step)`

Prüft, ob das aktuelle Intervall (step) erreicht ist.

Modellspeicherung: Nutzt torch.save, um die Gewichte (state_dict) der Agenten als .pth-Datei zu sichern. Dies ermöglicht es, das Training später exakt an diesem Punkt fortzusetzen oder den Agenten gegen Menschen spielen zu lassen.

Replay-Protokoll: Erstellt eine Textdatei (game_episode[n].txt), die den visuellen Verlauf der letzten Partie speichert.

`log_episode(self, session, win_rate_a, win_rate_b)`

Erfasst nach jeder Episode die aktuellen Werte für Epsilon, Belohnung und Loss.

Falls eine Evaluation stattgefunden hat, werden auch die neuen Siegquoten (win_rate) hinzugefügt.

Diese Datenwerden gesammelt, um am Ende einer Session als CSV-Datei exportiert zu werden.

`plot_overview(self, data_frame, name)`

Erzeugt eine längliche Übersicht mit mehreren Subplots (Loss-Kurve, Win-Rate-Entwicklung, Reward-Verlauf). Primär gedacht für volles Training.

Dateiformate: Speichert Grafiken sowohl als .png (für die schnelle Ansicht) als auch als .svg (Vektorgrafik für höchste Qualität).

`plot_comparison(self, data_a, fata_b)`

Die Methode nimmt Daten von verschiedenen Agenten und zeichnet deren Trainingsdaten untereinander um sie besser vergleichen zu können.

`plot(data_frame, name)`

Ähnlich zu plot_overview, aber Reward Graph ist höher, da größte Ausschwankungen auftreten. Primär für einzlne Sessions gedacht.

2.8 Training

Die Klasse Training stellt die Benutzeroberfläche für den Trainingsprozess dar. Sie dient als interaktiver Zugang zur Trainer-Klasse und ermöglicht es, ohne Code-Änderungen verschiedene Trainingsarten zu konfigurieren, Hyperparameter festzulegen und bestehende Modelle weiterzuentwickeln.

Klassen-Struktur & Initialisierung

Die Klasse ist darauf ausgelegt, den Benutzer durch einen logischen Entscheidungsprozess zu führen – vom Laden eines Modells bis hin zur Wahl der Trainingsintensität.

`__init__(self)`

Initialisiert die Instanz, besitzt zu Beginn jedoch noch keinen aktiven Trainer. Dieser wird erst durch die setup-Methode zugewiesen.

Zentrale Methoden

`setup(self)`

Dies ist der Einstiegspunkt für jede Sitzung:

Moduswahl: Der Nutzer entscheidet, ob ein völlig neuer Trainer (neue Verzeichnisstruktur) erstellt oder ein existierender Trainer aus dem training/-Ordner geladen werden soll.

Architektur-Konstanz: Beim Laden eines existierenden Trainers wird automatisch die korrekte hidden_size aus der Datei DQN_attributes gelesen. Dies verhindert Laufzeitfehler, die entstehen würden, wenn man versucht, Gewichte in eine falsch dimensionierte Netzwerk-Architektur zu laden.

`get_hyperparameters(self) -> dict`

Ermöglicht die Steuerung des Lernverhaltens:

Fragt Werte für gamma, epsilon (Start/Min/Decay) und etc. ab.

Für jeden Wert wird ein Standardwert (Default) vorgeschlagen. Bestätigt der Nutzer nur mit der Eingabetaste, wird dieser sichere Standardwert übernommen.

`select_training(self) -> str`

Präsentiert dem Nutzer dieVier bereits vorgestellten verfügbaren Trainings-Szenarien aus Trainer

`train(self)`

Die Steuerungslogik, die alle Informationen zusammenführt:

Führt das setup durch.

Fragt die gewünschte Trainingsform und die Episodenanzahl ab.

*Übergibt die gesammelten Hyperparameter (via Dictionary-Unpacking **hyperparameters) an die entsprechende Methode der Trainer-Klasse.*

2.9 Game

Die Klasse Game bildet die Oberfläche für den menschlichen Nutzer. Während die anderen Klassen primär dem Training dienen, ist die Game-Klasse dafür zuständig, die trainierten Modelle in einer spielbaren Umgebung gegen Menschen oder gegeneinander antreten zu lassen. Sie verwaltet die Spielinstanz, die Punkteübersicht und die Eingabeschleife.

Klassen-Struktur & Initialisierung

Die Klasse fungiert als Brücke, die sowohl VierGewinnt (Umgebung) als auch den Agent (KI) oder einfache Strings (für menschliche Spielernamen) aufnimmt.

`__init__(self, player1, player2)`

Spielfeld: Erstellt eine frische Instanz von VierGewinnt.

Spieler-Mapping: Nutzt ein Dictionary {1: player1, -1: player2}, um die internen numerischen Werte der Umgebung (1 und -1) flexibel den gewählten Spielertypen zuzuweisen.

Scoreboard: Initialisiert einen Zähler, um die Siege über mehrere Runden hinweg festzuhalten.

Zentrale Methoden

`select_player(self, player : str)`

Ermöglicht die Konfiguration der Partie vor dem Start:

Der Nutzer wählt pro Slot zwischen "KI" und "Mensch".

Bei Wahl der KI wird die Hilfsfunktion select_agent() aus logic.py aufgerufen, die ein Laden bestehender Modelle aus dem training/-Verzeichnis erlaubt.

Wichtig: Bei KI-Spielern wird epsilon automatisch auf 0.0 gesetzt, damit das Modell nur sein gelerntes Wissen nutzt und nicht mehr zufällig experimentiert.

`get_human_move(self) -> int`

Lässt Nutzer selbst einen legalen Zug wählen.

`run(self)`

Solange env.done falsch ist, wird abwechselnd ein Zug angefordert.

Die Methode prüft mittels isinstance(current_player, Agent), ob gerade eine KI am Zug ist oder ein Mensch. Entsprechend wird entweder agent.act() aufgerufen oder die Benutzereingabe abgefragt.

Nach jedem Zug wird das Spielfeld mit env.show_board() grafisch in der Konsole ausgegeben.

`play(self)`

Die übergeordnete Steuerung für eine Sitzung:

Erlaubt es, nach einem Sieg oder Unentschieden direkt eine "Neue Runde" zu starten, ohne das Programm neu laden zu müssen.

Aktualisiert nach jedem Spiel das Scoreboard und gibt den aktuellen Stand aus.

2.10 logic

Das Modul logic.py ist eine Sammlung von Hilfsfunktionen, die klassenübergreifend eingesetzt werden. Es dient der Behandlung wiederkehrender Aufgaben wie der Dateiverarbeitung, der Pfadnavigation und der Validierung von Benutzereingaben. Durch dieses Modul wird der Code in den Hauptklassen selbst übersichtlicher.

Zentrale Funktionen

`get_attributes_from_file(file) -> dict`

Diese Funktion ist essenziell für die Konsistenz des Projekts:

Sie liest Konfigurationsdateien (wie DQN_attributes) ein und extrahiert Atttribute. Sie stellt sicher, dass beim Laden eines gespeicherten Modells automatisch die richtige hidden_size für das neuronale Netz verwendet wird, wodurch Inkompatibilitäten beim Laden von Gewichten vermieden werden.

`select_path(path, prefix) -> str`

Ermöglicht eine komfortable Navigation im Dateisystem über das Terminal:

Scannt einen Ordner (z. B. training/) nach Unterordnern mit einem bestimmten Präfix (z. B. trainer_). Präsentiert dem Nutzer eine nummerierte Liste zur Auswahl.

Vorteil: Der Nutzer muss keine komplizierten Pfade manuell abtippen.

`select_agent() -> Agent`

Ein spezialisierter Workflow zum Laden einer KI:

Nutzt select_path, um zuerst den Trainer und dann das spezifische Modell innerhalb dieses Trainers auszuwählen.

Liest die Netzwerk-Attribute aus.

Erzeugt eine neue Agent-Instanz mit der korrekten Architektur und lädt die gespeicherten Gewichte (.pth-Datei).

`next_directory(directory, prefix) -> str`

Prüft, welche Ordner mit dem gewählten Präfix bereits existieren.

Generiert den nächsten freien Namen nach dem Schema name_n+1 und verhindert somit das versehentliche Überschreiben von wertvollen Trainingsdaten bei neuen Versuchen.

`select_value(prompt, default, value_type) -> Any`

Erlaubt die Festlegung von Standardwerten (Defaults), die durch Drücken von Enter übernommen werden.

Überprüft, ob die Eingabe dem geforderten Datentyp (int oder float) entspricht, und verlangt bei Fehlern eine neue Eingabe.

2.11 main

Die Datei main.py ist das zentrale Steuerungsmodul des Programms. Sie enthält keine komplexe Logik, sondern startet die Benutzerinteraktion und ruft die entsprechenden Klassen (Training oder Game) auf.

```
main()
```

Dies ist die Hauptfunktion, die beim Programmstart ausgeführt wird:

Begrüßung & Moduswahl: Der Nutzer wird gefragt, ob er eine KI trainieren oder eine Partie spielen möchte.

Verzweigung:

Option 1 (Training): Erstellt eine Instanz der Klasse Training und startet deren interaktiven Setup-Prozess.

Option 2 (Spielen): Erstellt eine Instanz der Klasse Game. Hier kann der Nutzer dann wählen, ob er gegen eine geladene KI oder einen anderen Menschen spielt.

Nach beenden der jeweiligen Aktion stoppt das Programm sauber

3 Quellen

Dokumentationen der hauptsächlich verwendeten Bibliotheken:

Matplotlib (2026): Matplotlib 3.10.8 documentation, URL: https://matplotlib.org/stable/index.html [Zuletzt aufgerufen am 12.03.2026].

NumPy (2026): NumPy Dokumentation: https://numpy.org/doc/ [Zuletzt aufgerufen am 11.03.2026].

Pandas (2026): Pandas Dokumentation. URL: https://pandas.pydata.org/docs/index.html [Zuletzt aufgerufen am 12.03.2026].

Sonstige:

GeeksforGeeks (2026): Python - Passing Dictionary as Arguments to Function, URL: https://www.geeksforgeeks.org/python/python-passing-dictionary-as-arguments-to-function/ [Zuletzt aufgerufen am 9.03.2026].

Leapcell (2026): Understanding Optional Arguments in Python, URL: https://leapcell.io/blog/understanding-optional-arguments-in-python [Zuletzt aufgerufen am 21.02.2026].

Python Software Foundation (2026): Python Documentation – copy (Shallow and deep copy operations), URL: https://docs.python.org/3/library/copy.html [Zuletzt aufgerufen am 27.02.2026].

StackOverflow (2026): Does Python have a string 'contains' substring method?, URL: https:// stackoverflow.com/questions/3437059/does-python-have-a-string-contains-substring-method [Zuletzt aufgerufen am 7.03.2026].

StackOverflow (2026): How do I define a function with optional arguments?, URL: https://stackoverflow.com/questions/9539921/how-do-i-define-a-function-with-optional-arguments [Zuletzt aufgerufen am 21.02.2026].

StackOverflow (2026): Putting a simple if-then-else statement on one line, URL: https://stackoverflow.com/questions/2802726/putting-a-simple-if-then-else-statement-on-one-line [Zuletzt aufgerufen am 1.03.2026].

StackOverflow (2026): Understanding Python super() with init() methods, URL: https://stackoverflow.com/questions/576169/understanding-python-super-with-init-methods [Zuletzt aufgerufen am 15.02.2026].

W3Schools (2026): Python Dictionaries - Change Items, URL: https://www.w3schools.com/python/python_dictionaries_change.asp [Zuletzt aufgerufen am 21.02.2026].

W3Schools (2026): Python Random choice() Method, URL: https://www.w3schools.com/python/ref_random_choice.asp [Zuletzt aufgerufen am 15.02.2026].

W3Schools (2026): Python Try Except, URL: https://www.w3schools.com/python/python_try_except.asp [Zuletzt aufgerufen am 27.02.2026].